

```

198.         boardAfterMove[secondTileSpace] = combinedTilesColumn[1]
199.         boardAfterMove[thirdTileSpace] = combinedTilesColumn[2]
200.         boardAfterMove[fourthTileSpace] = combinedTilesColumn[3]
201.
202.     return boardAfterMove
203.
204.
205. def askForPlayerMove():
206.     """Asks the player for the direction of their next move (or quit).
207.
208.     Ensures they enter a valid move: either 'W', 'A', 'S' or 'D'. """
209.     print('Enter move: (WASD or Q to quit)')
210.     while True: # Keep looping until they enter a valid move.
211.         move = input('> ').upper()
212.         if move == 'Q':
213.             # End the program:
214.             print('Thanks for playing!')
215.             sys.exit()
216.
217.         # Either return the valid move, or loop back and ask again:
218.         if move in ('W', 'A', 'S', 'D'):
219.             return move
220.         else:
221.             print('Enter one of "W", "A", "S", "D", or "Q".')
222.
223.
224. def addTwoToBoard(board):
225.     """Adds a new 2 tile randomly to the board."""
226.     while True:
227.         randomSpace = (random.randint(0, 3), random.randint(0, 3))
228.         if board[randomSpace] == BLANK:
229.             board[randomSpace] = 2
230.             return # Return after finding one non-blank tile.
231.
232.
233. def isFull(board):
234.     """Returns True if the board data structure has no blanks."""
235.     # Loop over every space on the board:
236.     for x in range(4):
237.         for y in range(4):
238.             # If a space is blank, return False:
239.             if board[(x, y)] == BLANK:
240.                 return False
241.     return True # No space is blank, so return True.
242.
243.
244. # If this program was run (instead of imported), run the game:
245. if __name__ == '__main__':
246.     try:
247.         main()
248.     except KeyboardInterrupt:
249.         sys.exit() # When Ctrl-C is pressed, end the program.

```
